



SYMBIOSIS INSTITUTE OF TECHNOLOGY, PUNE

Constituent of Symbiosis International (Deemed University), Pune

Name: Devansh Singh
PRN: 25070122072
Batch: A-2
Division: A

Subject: Programming with Java

ASSIGNMENT No : 28

TITLE : Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication.

Problem Statement

Develop a Java application demonstrating Remote Method Invocation (RMI) using stub and skeleton components for remote communication.

Learning Objectives

To implement Remote Method Invocation for distributed Java applications.

Theory

Remote Method Invocation (RMI) is a Java technology that enables an object running on one Java Virtual Machine (JVM) to invoke methods of an object running on another JVM located on a different machine.

RMI simplifies distributed application development by allowing remote objects to behave like local objects. It consists of remote interfaces, remote object implementations, the RMI registry, and client-server communication.

The registry maintains references to remote objects, allowing clients to locate and invoke them. RMI provides platform-independent communication, object serialization, and automatic network handling, making it suitable for distributed enterprise applications.

Output Screenshots

```
• dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp28/Main.java
RMI Server is running...
Hello from RMI Server
❖ dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$
```

CODE

```
1 package Exp28;
2
3 import java.rmi.*;
4 import java.rmi.server.*;
5 import java.rmi.registry.*;
6
7 interface Hello extends Remote {
8     String message() throws RemoteException;
9 }
10
11 class HelloImpl extends UnicastRemoteObject implements Hello {
12
13     HelloImpl() throws RemoteException {
14         super();
15     }
16
17     public String message() throws RemoteException {
18         return "Hello from RMI Server";
19     }
20 }
21
22 public class Main {
23     public static void main(String[] args) {
24         try {
25             LocateRegistry.createRegistry(1099);
26
27             HelloImpl obj = new HelloImpl();
28
29             Naming.rebind("HelloService", obj);
30
31             System.out.println("RMI Server is running...");
32             System.out.println(obj.message());
33
34             System.exit(0);
35         }
36     }
37 }
```

```

36         catch (Exception e) {
37             System.out.println("Error: " + e.getMessage());
38         }
39     }
40 }

```

EXERCISE 1

```

1 package Exp28.Exercise;
2
3 import java.rmi.*;
4 import java.rmi.server.*;
5 import java.rmi.registry.*;
6
7 interface Calculator extends Remote {
8     int add(int a, int b) throws RemoteException;
9     int subtract(int a, int b) throws RemoteException;
10    int multiply(int a, int b) throws RemoteException;
11    int divide(int a, int b) throws RemoteException;
12 }
13
14 class CalculatorImpl extends UnicastRemoteObject implements Calculator {
15
16     CalculatorImpl() throws RemoteException {
17         super();
18     }
19
20     public int add(int a, int b) throws RemoteException {
21         return a + b;
22     }
23
24     public int subtract(int a, int b) throws RemoteException {
25         return a - b;
26     }
27
28     public int multiply(int a, int b) throws RemoteException {
29         return a * b;
30     }
31
32     public int divide(int a, int b) throws RemoteException {
33         return a / b;
34     }
35 }
36
37 public class Exp1 {

```

```

38     public static void main(String[] args) {
39         try {
40             // Start RMI Registry
41             LocateRegistry.createRegistry(1099);
42
43             // Create remote object
44             CalculatorImpl obj = new CalculatorImpl();
45
46             // Register remote object
47             Naming.rebind("CalculatorService", obj);
48
49             System.out.println("Calculator RMI Server is running...");
50
51             System.out.println("Addition: " + obj.add(20, 10));
52             System.out.println("Subtraction: " + obj.subtract(20, 10));
53             System.out.println("Multiplication: " + obj.multiply(20, 10));
54             System.out.println("Division: " + obj.divide(20, 10));
55
56             System.exit(0);
57         }
58         catch (Exception e) {
59             System.out.println("Error: " + e.getMessage());
60         }
61     }
62 }

```

EXERCISE 2

```

1  package Exp28.Exercise;
2
3  import java.rmi.*;
4  import java.rmi.server.*;
5  import java.rmi.registry.*;
6
7  interface Temperature extends Remote {
8      double celsiusToFahrenheit(double celsius)
9          throws RemoteException;
10
11     double fahrenheitToCelsius(double fahrenheit)
12         throws RemoteException;
13 }
14
15 class TemperatureImpl extends UnicastRemoteObject
16     implements Temperature {
17

```

```

18     TemperatureImpl() throws RemoteException {
19         super();
20     }
21
22     public double celsiusToFahrenheit(double celsius)
23         throws RemoteException {
24         return (celsius * 9 / 5) + 32;
25     }
26
27     public double fahrenheitToCelsius(double fahrenheit)
28         throws RemoteException {
29         return (fahrenheit - 32) * 5 / 9;
30     }
31 }
32
33 public class Exp2 {
34     public static void main(String[] args) {
35         try {
36             // Start RMI Registry
37             LocateRegistry.createRegistry(1099);
38
39             // Create remote object
40             TemperatureImpl obj = new TemperatureImpl();
41
42             // Register remote object
43             Naming.rebind("TemperatureService", obj);
44
45             System.out.println("Temperature RMI Server is running...");
46
47             double celsius = 25;
48
49             double fahrenheit =
50                 obj.celsiusToFahrenheit(celsius);
51
52             System.out.println(
53                 celsius + " Celsius = "
54                 + fahrenheit + " Fahrenheit"
55             );
56
57             System.out.println(
58                 fahrenheit + " Fahrenheit = "
59                 + obj.fahrenheitToCelsius(fahrenheit)
60                 + " Celsius"
61             );
62

```

```

63         System.exit(0);
64     }
65     catch (Exception e) {
66         System.out.println("Error: " + e.getMessage());
67     }
68 }
69 }

```

Exercise

Q1. Create a remote calculator service using Java RMI.

```

• dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp28/Exercise/Exp1.java
Calculator RMI Server is running...
Addition: 30
Subtraction: 10
Multiplication: 200
Division: 2
• dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$

```

Q2. Create a Java application using RMI where the server converts temperature values between Celsius and Fahrenheit. The client sends input temperature and receives converted output.

```

• dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$ java Exp28/Exercise/Exp2.java
Temperature RMI Server is running...
25.0 Celsius = 77.0 Fahrenheit
77.0 Fahrenheit = 25.0 Celsius
• dslord@Fedora:~/Desktop/Coding Projects/Java/Java Lab$

```

Conclusion

Thus, the experiment successfully demonstrated the concept of Remote Method Invocation (RMI) in Java for distributed communication. The role of remote interfaces, remote objects, and the RMI registry was understood. RMI enables a client to invoke methods of an object running on another JVM. The experiment provided an understanding of remote communication and distributed Java applications.